

DocFlow

SECURITY
DESIGN

Security Design Review

PURPOSE	Architecture and security design input for AWS Security Agent
SYSTEM	DocFlow document-management and OCR application
BASIS	Repository source and deployment files inspected 2026-09-22
STATUS	Current implementation with deployment assumptions explicitly labeled

Handling notice: Security-sensitive architecture details. Credentials, secret values, real document contents, and live personal records are intentionally excluded.

1. Executive summary

DocFlow manages university appointment-order documents. Users upload or import PDF, DOCX, XLSX, and CSV files. A Go API stores originals on a persistent filesystem, extracts content locally, and stores metadata, OCR output, personnel mappings, revision history, and audit records in MySQL. A Next.js web application exposes the user interface and proxies browser requests to the API.

The intended production entry point is a Cloudflared tunnel to the web container. The API trusts upstream identity and role headers. The repository requires a university SSO or trusted reverse proxy to authenticate the user, remove client-supplied identity headers, and inject verified values; however, that authenticating component and its configuration are not included in this repository. This is the primary trust-boundary assumption for the design review.

The design processes untrusted documents with Poppler, Python, ZIP/XML parsers, and a local OCR model. It also stores personal and professional directory data. Important review areas are identity-header integrity, unauthorized read access, resource exhaustion during document parsing, active-content delivery, retention enforcement, audit durability, secret handling, transport security, and backup/restore controls.

2. Scope and review objectives

In scope

- Next.js web application and `/backend/*` proxy.
- Go/Gin REST API, authentication middleware, authorization checks, and scheduled jobs.
- MySQL metadata store and persistent document-file storage.
- Browser uploads and scheduled folder imports.
- PDF rendering, OneOCR processing, and structured DOCX/XLSX/CSV extraction.
- University directory synchronization using a bearer API key.
- Optional SMTP expiry notifications.
- Docker development topology and Podman/Cloudflared production topology supplied by the repository.

Out of scope or not verified

- Live Cloudflared tunnel, university SSO, DNS, TLS, firewall, and host configuration.
- Live IAM, cloud account, volume encryption, and backup configuration.
- Security properties of the external university users API and SMTP server.
- Runtime penetration testing and validation against a deployed environment.
- Security of third-party OCR model files beyond repository configuration.

Review objectives

1. Verify that only authenticated university users can reach application data.
2. Verify that roles cannot be forged and that mutations are server-authorized.
3. Contain untrusted document parsing and resource consumption.
4. Protect stored documents, OCR text, directory data, secrets, and audit evidence.
5. Ensure production deployment fails closed and remains observable and recoverable.

3. System context

Primary actors

- VIEWER: authenticated user intended to read/search documents and directory data.
- STAFF: may upload, replace, edit, confirm documents, create directory records, and trigger user synchronization.
- ADMIN: receives STAFF mutation capabilities and may delete documents.
- DEVELOPER: may delete documents but is not otherwise granted STAFF mutations by route configuration.
- Folder import service: system actor importing files from a mounted incoming directory.

- Scheduled jobs: synchronize users, create/send expiry notifications, and scan the import folder.
- External university directory: returns personnel records to the API.
- SMTP server: receives notification messages for matched personnel.

High-level architecture



4. Deployment topology and trust boundaries

Production topology described by repository

- MySQL and the API join docflow-internal.
- The web joins docflow-internal and external proxy-net.
- Production Compose does not publish host ports for MySQL, API, or web.
- The deployment guide routes Cloudflared to `http://docflow-web:3000`.
- The network named docflow-internal is not declared with Compose `internal: true`; its name alone does not prove egress isolation.
- Cloudflared, university SSO, header sanitization, TLS policy, host firewall rules, and network policy are deployment assumptions rather than repository-provided controls.

Development topology

- MySQL, API, and web publish host ports 3307, 8080, and 3000.
- Development authentication defaults to disabled.
- Development database credentials are non-production literal defaults.
- Development behavior must not be promoted unchanged into production.

Trust boundaries

1. Browser to ingress: hostile clients can send arbitrary HTTP headers and bodies.
2. Ingress to web/API: identity headers are trusted only if ingress strips caller values and injects verified values.
3. API to document parsers: every document is untrusted content even after extension and magic-byte checks.
4. API to database and file volume: contains sensitive institutional documents and personal data.
5. API to university directory: outbound bearer credential and a large personal-data response.
6. API to SMTP: sends recipient addresses and appointment/order details.
7. Operator and backup boundary: administrative access and backup execution are not implemented by application code.

5. Identity and access design

The API applies authentication middleware globally before registering health and application routes. The middleware reads configurable request headers, defaulting to `X-Auth-Request-User` and `X-Auth-Request-Role`.

When `AUTH_REQUIRED=true`, an empty user header is rejected. A nonempty value is treated as an authenticated identity without cryptographic verification, issuer/audience checking, session validation, directory lookup, or trusted-proxy verification in the application. A missing role becomes VIEWER. When authentication is disabled, the fallback user is `local-user` and the fallback role is ADMIN.

Mutation authorization is route based:

Operation	Allowed roles
Upload, replace, edit, confirm document	ADMIN, STAFF
Create directory person	ADMIN, STAFF
Trigger directory synchronization	ADMIN, STAFF
Delete document	ADMIN, DEVELOPER
Read/search documents, files, OCR images, audits, directory, reports	Any request accepted by authentication middleware

There is no document-owner authorization or enforcement of the stored confidentiality field in the inspected read handlers. Unknown but nonempty roles pass authentication and can access read endpoints when the identity header is accepted.

Required identity security invariant

Production must accept identity and role headers only from an authenticated trusted ingress that removes all client-supplied values before injecting verified values. Direct API bypass must be prevented. Alternatively, the API must independently verify signed identity assertions. Anonymous or forged-header requests must not read application data or perform mutations.

6. Data assets and classification

Asset	Examples	Proposed handling classification
Original documents	appointment orders, filenames, signatures and embedded content	Confidential institutional data
Extracted/OCR data	raw text, names, positions, responsibilities, dates, bounding boxes	Confidential institutional and personal data
Directory records	name, gender, email, phone, position, room, researcher IDs, raw upstream payload	Personal data
Identity/audit data	imported-by, confirmed-by, revisions, actions, timestamps	Security and accountability data
Secrets	DB credentials, API key, SMTP credentials, tunnel credentials	Restricted secrets
Operational data	processing errors, sync runs, expiry notifications	Internal operational data; errors may contain document-derived details

The JSON API suppresses storage paths, directory raw payload/fingerprint, and OCR raw JSON, while returning much of the directory personal data to authenticated read users.

7. Data flows

7.1 Browser upload and processing

1. An ADMIN or STAFF request submits multipart field `file` through the web proxy.
2. The API allows PDF, DOCX, XLSX, and CSV and checks a declared maximum file size of 50 MiB.

3. The API computes SHA-256 for duplicate detection.
4. PDF receives a `%PDF-` prefix check; DOCX/XLSX receive a ZIP magic check; CSV receives no signature validation.
5. The API creates a timestamp-plus-basename storage path and writes the original into the upload volume.
6. Metadata and a ten-year `retention_until` timestamp are written to MySQL.
7. In-process background work renders/imports the document and stores derived data in MySQL.
8. Staff review and confirmation create audit data. Some audit-write errors are ignored rather than failing the user operation.

7.2 PDF processing

1. `pdftinfo` is used to detect page count; a detected count over 100 is rejected.
2. `pdftoppm` renders pages to temporary PNG files.
3. Python OneOCR reads the images and writes structured JSON.
4. The Go service stores pages, lines, fields, appointments, confidence, and processing results.
5. Temporary processing directories are removed when processing returns.

If `pdftinfo` fails, the helper returns zero and the subsequent render is not given a final-page bound. Processing jobs use in-process goroutines without a queue/concurrency limit, explicit processing deadline, or restart recovery in the inspected implementation.

7.3 Structured document processing

CSV is read as text. DOCX and XLSX are ZIP containers whose XML entries are decompressed and parsed in memory. The inspected parser has no decompressed-size or archive-entry-count limit. Office applications and macros are not executed by this parser.

7.4 Scheduled folder import

The API scans a mounted incoming directory at startup and on a configurable schedule, defaulting to every five minutes. It skips unsupported extensions and recently modified files, then applies size, signature, and hash checks before copying the file to managed storage. Imported source files remain in the incoming directory.

7.5 University directory synchronization

The API performs an outbound GET to a configured university endpoint using `Authorization: Bearer ${API_KEY}`. The client has a 60-second timeout and bounds the response to 20 MiB. Returned users are upserted and the original per-user JSON is retained in MySQL. Missing users from a later response are not deleted or deactivated by the synchronization loop.

7.6 Expiry notification email

A scheduled job creates notifications for documents whose expiry dates fall in configured future windows. If SMTP is configured, the API sends the matched directory email address plus appointment/order details. The application uses Go SMTP with optional plain authentication; mandatory TLS is not configured explicitly in application code.

8. Persistence, retention, audit, and recovery

- Original files are stored in the upload filesystem volume, not in MySQL.
- MySQL stores metadata, OCR output, personnel data, revisions, audits, notifications, aliases, and synchronization history.
- SQL migrations are embedded and run using the runtime database connection, so the runtime DB identity needs migration DDL privileges.
- Upload/import/reupload sets `retention_until` to ten years from the operation.
- No automatic purge, legal hold, retention-policy enforcement, or retention-based delete guard was found.
- An authorized delete can remove the document, derived rows, revision history, audit records, and stored original before `retention_until`.
- A folder-import source remains in the incoming directory and may be imported again after deletion.

- The deployment guide documents manual MySQL dump and states that document files require an independent backup. No automated encrypted backup, retention schedule, or restore verification is supplied.
- Audit data is stored in the same mutable database and can be deleted with the document; no external immutable audit sink is configured.

9. Implemented security controls

- Production manifest defaults `AUTH_REQUIRED` to true.
- Server-side role checks protect mutation endpoints.
- Production topology does not publish service host ports.
- API container runs as a non-root `app` user.
- Fixed configured CORS origin and credentialed browser requests.
- Extension allowlist, 50 MiB declared size check, limited magic-byte validation, SHA-256 duplicate detection, timestamped basename storage.
- PDF detected-page limit and post-processing appointment-count limit.
- External directory request timeout and response-size bound.
- SQL queries use GORM parameters for inspected search filters.
- DSN log output redacts the password and logging reports only whether the external API key is configured.
- Transactional processing/update audit entries and revision records for several document fields.
- Secrets are expected through environment configuration and `.env*` is excluded by the root Docker build context.

10. Design gaps and assumptions requiring review

Highest-priority release gates

1. Identity-header integrity is assumed. The repository does not provide SSO verification or header sanitization. The deployment guide includes an example that sends ADMIN headers from the client. Production requires a verified ingress design and a direct-bypass test.
2. Authenticated health check conflict. Global authentication protects `/health`, but the production container health probe sends no identity header while production defaults authentication on. The code path predicts HTTP 401 and an unhealthy container unless external configuration differs.
3. Sensitive reads lack fine-grained authorization. Any accepted identity, including an unknown role, can reach document files, OCR output, audit logs, directory PII, search, and reports. Confidentiality classification is stored but not enforced in inspected handlers.
4. Untrusted document resource controls are incomplete. No decompressed-size limit for Office ZIP content, no bounded processing queue, no processing deadline, and page bounding can be bypassed when `pdfinfo` fails.
5. Uploaded response type is partly client controlled. Multipart MIME is stored and replayed as the inline response `Content-Type`; CSV receives no content signature validation. Server-selected safe MIME and download policy are required.

Additional gaps

- No CSRF protection, request rate limiting, total request-body cap, explicit HTTP server timeouts, or application security-header policy was found.
- CORS allows the identity headers; CORS is not an authentication control.
- SMTP transport does not explicitly require TLS in application configuration.
- Retention is recorded but not enforced, while delete removes revision and audit evidence.
- Some audit write failures are ignored.
- Production loads the full `.env` into the API; a shared documented environment also contains the MySQL root password.
- No container resource limits, read-only filesystem, or capability drops are supplied.
- Web container does not declare a non-root runtime user in its Dockerfile.

- No automated backup, encryption, restore test, or immutable audit destination is included.
- Live TLS termination, disk encryption, firewall rules, tunnel access policy, and monitoring are unverified deployment assumptions.

11. Required security requirements for the review

SR-1: Prevent forged SSO identity and role headers

Production must require authentication, accept identity and role only from a trusted authenticated ingress that strips caller-supplied values, prevent direct API bypass, and enforce server-side roles. Integration tests must cover missing identity, forged headers through `/backend`, direct API access, unknown roles, and role denial.

SR-2: Restrict sensitive document and directory reads

Every read of original files, OCR pages/text, directory personal data, audit logs, search, and reports must require an authorized application role. Where document confidentiality or organizational scope applies, authorization must be enforced server side for each resource.

SR-3: Safely process untrusted documents

All document formats must be validated using server-selected types and processed with bounded compressed and decompressed size, page count, entry count, memory, CPU, concurrency, and execution time. Parser processes must be isolated and failures must not expose sensitive content.

SR-4: Enforce retention and durable audit policy

Retention decisions must be enforced rather than represented only by a timestamp. Deletion before the retention date must follow an explicit authorized exception, and accountability records required by policy must survive document deletion in a protected audit destination.

SR-5: Protect secrets and external communications

Production secrets must not be stored in source, image layers, logs, client code, or broadly shared environment files. Database, directory API, SMTP, ingress, and backup communications must use authenticated encrypted transport appropriate to their trust boundary.

SR-6: Fail closed and remain recoverable

Production startup and health checks must work without bypassing authentication controls. The service must expose only intended network paths, apply operational resource limits, back up both MySQL and document files, encrypt backups, and regularly verify restoration.

12. Security validation plan

- Send anonymous and forged identity/role requests through the public `/backend` path; verify ingress removes values and the API rejects the request.
- Attempt direct access to the API network endpoint from outside the trusted web/ingress path.
- Test VIEWER and unknown roles against every mutation and sensitive read endpoint.
- Verify STAFF cannot delete and DEVELOPER does not gain unrelated STAFF mutations.
- Upload mislabeled content, malformed PDF, oversized PDF, high-page PDF, ZIP bomb, excessive-entry DOCX/XLSX, large CSV, and duplicate files in an isolated test environment.
- Verify server-selected response MIME and safe download/display behavior.
- Confirm container health with production authentication enabled.
- Simulate OCR worker failure, restart during processing, external API timeout, SMTP failure, audit-write failure, and full storage.
- Verify delete/retention/legal-hold behavior and confirm required audit evidence remains available.
- Inspect the deployed tunnel policy, TLS termination, header transforms, firewall, published ports, volume encryption, secret distribution, monitoring, backup jobs, and restore evidence.

13. Open questions for system owners

1. Which component authenticates the university session, and how does it strip and inject identity and role headers?
2. Can any user or network path reach the API without passing through that component?
3. Which roles may view original documents, directory PII, audits, and confidential documents?
4. Is the ten-year period a minimum retention, maximum retention, or default review date?
5. Must audit history survive an authorized document deletion?
6. What are the approved encryption, backup, restore-time, and recovery-point requirements?
7. Are SMTP and the university API required to use validated TLS, mTLS, or IP allowlisting?
8. What document parser isolation and availability limits are acceptable for production?
9. How are inactive or removed directory users reflected locally and in application access?
10. What monitoring system receives authentication failures, privileged actions, parser failures, and backup alerts?

14. Evidence index

Claims in this document were confirmed from the following repository paths:

- `backend/auth.go`: identity-header authentication and role fallback.
- `backend/main.go`: domain models, routes, upload controls, schedules, CORS, audit creation.
- `backend/api_handlers.go`: document reads, updates, delete, replace, search, response behavior.
- `backend/document_processing.go`: background queue, PDF/OneOCR pipeline, limits and audit.
- `backend/ocr_worker.py`: CSV and Office structured extraction.
- `backend/file_validation.go`: file signature validation.
- `backend/folder_import.go`: incoming-directory import flow.
- `backend/sync.go`: university directory request, bounds, and upsert.
- `backend/expiry.go`: expiry notification and SMTP behavior.
- `backend/migrations/*.sql`: persisted entities and indexes.
- `src/lib/api.ts`: browser API base path and request behavior.
- `next.config.ts`: `/backend/*` rewrite.
- `docker-compose.yml`: development topology.
- `deploy/podman-compose.yml`: production topology and defaults.
- `backend/Dockerfile` and `Dockerfile`: container build and runtime users.
- `README.md` and `DEPLOY_PODMAN.md`: documented operating assumptions.

15. Reconciled documentation conflicts

- A README statement can be read as storing originals in MySQL. Implementation writes originals to the filesystem volume and stores their paths and derived data in MySQL. This document follows implementation.
- Documentation calls the production network internal. The manifest names it `docflow-internal` but does not set the Compose `internal` attribute. This document treats it as topology, not proven isolation.
- Documentation describes ten-year retention. Implementation assigns a date but does not enforce retention or legal hold. This document distinguishes metadata from policy enforcement.
- A README confirmation example includes `userId`, but implementation attributes confirmation to middleware identity and the frontend sends an empty body. This document follows implementation.

Handling notice: This design contains security-sensitive architecture details but intentionally excludes credentials, secret values, real document contents, and live personal records. Share it only with authorized reviewers and the approved AWS Security Agent workspace.